Delta/Range Comparisons

Matcher	Passes if…	Available aliases
be_between(1, 10).inclusive	a >= 1 && a <= 10	be_between(1, 10) a_value_between(1, 10).inclusive a_value_between(1, 10)
be_between(1, 10).exclusive	a > 1 && a < 10	a_value_between(1, 10).exclusive
be_within(0.1).of(x)	(a - x).abs <= 0.1	a_value_within(0.1).of(x)
be_within(5).percent_of(x)	(a - x).abs <= (0.05 * x)	a_value_within(5).percent_of(x)
cover(x, y)	a.cover?(x) && a.cover?(y)	a_range_covering(x, y)

Strings and Collections

Matcher	Passes if…	Available aliases
contain_exactly(2, 1, 3)	a.sort == [2, 1, 3].sort	match_array([2, 1, 3]) a_collection_containing_exactly(2, 1, 3)
start_with(x, y)	a[0] == x && a[1] == y	a_collection_starting_with(x, y) a_string_starting_with(x, y)
end_with(x, y)	a[-1] == x && a[-2] == y	a_collection_starting_with(x, y) a_string_starting_with(x, y)
include(x, y)	(a.include?(x) && a.include?(y)) || (a.key?(x) && a.key?(y))	a_collection_including(x, y) a_string_including(x, y) a_hash_including(x, y)
include(w: x, y: z)	a[:w] == :x && a[:y] == :z	a_hash_including(w: x, y: z)
all(matcher)	a.all? { |e| matcher.matches?(e) }	
match(x: matcher, y: 3)	matcher.matches?(a[:x]) && a[:y] == 3	an_object_matching(x: matcher, y: 3)
match([3, matcher])	a[0] == 3 && matcher.matches?(a[1])	an_object_matching([3, matcher])
match("pattern")	a.match("pattern")	a_string_matching("pattern")
match(/regex/)	a.match(/regex/)	match_regex(/regex/) a_string_matching(/regex/)